

Revisão ENADE — Banco de Dados

Do modelo relacional ao NoSQL

Tópicos Especiais II — ENADE · CC e SI · 2026/2

Prof. M.Sc. Howard Cruz Roatti

Como funciona esta revisão

1. Recaptura relâmpago

- Os conceitos-chave de **cada unidade**, em uma passada rápida.
- Foco no que o **ENADE cobra**.

2. 🔥 Esquenta

- Questões **estilo ENADE** (situação-problema → A–D).
- Responda **antes** do gabarito comentado.

💡 A prova cobra **análise**, não decoreba: leia o contexto, elimine distratores, decida.

Parte 1

Recaptura relâmpago por unidade

Arquitetura & Modelo Relacional

- **ANSI-SPARC (3 níveis):** externo (views) · conceitual (esquema lógico) · interno (físico).
 - **Independência física** = mudar armazenamento/índices sem afetar aplicações.
 - **Independência lógica** = mudar o esquema conceitual sem afetar as views.
- **Modelo relacional (Codd):** dados em **relações (tabelas)**; **chave primária** (única, não-nula) e **chave estrangeira** (integridade referencial).
- **Álgebra relacional:** σ (seleção = filtra linhas) · π (projeção = escolhe colunas) · $\bowtie$ (junção) · $\cup, -, \times, \div$.

Modelagem & Normalização

- **MER:** entidades, atributos, relacionamentos, **cardinalidade**; **generalização/especialização** (total×parcial, disjunta×sobreposta); **entidade associativa** para **N:N**.
- **Normalização** (remove redundância e anomalias):
 - **1FN** — atributos **atômicos** (sem multivalorado/repetição).
 - **2FN** — 1FN + **sem dependência parcial** (não-chave dependendo de **parte** de chave composta).
 - **3FN** — 2FN + **sem dependência transitiva** (não-chave → não-chave).

💡 Dependência **parcial** → 2FN; dependência **transitiva** → 3FN.

SQL essencial

- **DDL** (CREATE/ALTER/DROP) · **DML** (INSERT/UPDATE/DELETE) · **DQL** (SELECT).
- **WHERE** × **HAVING** : **WHERE** filtra **linhas antes** de agrupar (sem agregação); **HAVING** filtra **grupos depois** do **GROUP BY** (com **SUM**, **COUNT** ...).
- **JOIN** combina tabelas pelas chaves; **GROUP BY** agrega por grupo.
- **Índices** aceleram leitura **seletiva** (poucas linhas), mas oneram escrita/espço.
- **View** = consulta nomeada (tabela virtual); **integridade referencial** via FK (**ON DELETE CASCADE/RESTRICT**).

Transações & NoSQL

- **ACID:** Atomicidade · Consistência · Isolamento · Durabilidade.
- **Anomalias de isolamento:** *dirty read* (lê não-confirmado) · *non-repeatable* (relê e muda) · *phantom* (novas linhas) · *lost update*.
- **Concorrência:** 2PL garante serialização mas **não evita deadlock** (espera circular → vítima + rollback).
- **Recuperação (WAL):** REDO nas confirmadas, UNDO nas não-confirmadas; **checkpoint**.
- **NoSQL:** chave-valor, documento, coluna, grafo; **Teorema CAP** (sob partição, escolha C ou A) e **BASE** (consistência eventual).

Parte 2 — Esquenta

Banco de Dados · questões estilo ENADE (A–D)

Responda antes de virar o slide!

Q1 — Álgebra relacional

Uma secretaria acadêmica usa `Aluno(Matricula, Nome, IdCurso)` e `Curso(IdCurso, NomeCurso)`. É preciso emitir uma lista com **somente os nomes** dos alunos do curso de **Sistemas de Informação**.

Assinale a expressão de álgebra relacional correta.

- A) $\pi_{\text{Nome}}(\sigma_{\text{NomeCurso}='Sistemas de Informação'}(\text{Aluno} \bowtie \text{Curso}))$
- B) $\pi_{\text{Nome}}(\text{Aluno} \bowtie \text{Curso})$
- C) $\sigma_{\text{NomeCurso}='Sistemas de Informação'}(\text{Aluno} \bowtie \text{Curso})$
- D) $\pi_{\text{NomeCurso}}(\sigma_{\text{Nome}='Sistemas de Informação'}(\text{Aluno} \bowtie \text{Curso}))$

Q1 — Gabarito: A

Relembrando: σ filtra **linhas**; π escolhe **colunas**; a junção casa `IdCurso`.

- Precisa **filtrar** pelo curso (σ) e **projetar** só o `Nome` (π). 
- **B** não filtra (traz todos); **C** filtra mas **não projeta** (devolve a tupla inteira); **D** usa atributos trocados.

Q2 — SQL com agregação

Em `Cliente(id, nome)`, `Pedido(id, id_cliente)`, `Item(id_pedido, qtd, valor)`, o marketing quer os **clientes cujo total gasto** ($\sum \text{qtd} * \text{valor}$) **passa de R\$ 1.000**.

Qual consulta atende?

- A) `WHERE SUM(qtd*valor) > 1000` com `GROUP BY nome`.
- B) junta as três tabelas, agrupa por cliente e usa `HAVING SUM(qtd*valor) > 1000`.
- C) agrupa por cliente e usa `HAVING qtd*valor > 1000`.
- D) `WHERE valor > 1000`, item a item.

Q2 — Gabarito: B

Relembrando: filtro sobre **agregado por grupo** → `GROUP BY` + `HAVING` (o `WHERE` não aceita `SUM`).

- **A** usa `SUM` no `WHERE` (inválido); **C** compara item a item, não o total; **D** filtra itens caros, não o **gasto total** do cliente.  = **B**.

Q3 — Normalização

Um analista modela $R(\text{Aluno}, \text{Disciplina}, \text{Professor})$ com chave $(\text{Aluno}, \text{Disciplina})$ e a regra $\text{Disciplina} \rightarrow \text{Professor}$. Ele nota **redundância**: o professor se repete em cada matrícula.

A relação **viola** a:

- A) 1FN, por atributo multivalorado.
- B) 3FN, por dependência transitiva.
- C) 2FN, por dependência parcial de Professor em relação à chave.
- D) Nenhuma; já está na 3FN.

Q3 — Gabarito: C

Relembrando: Professor depende só de Disciplina (parte da chave composta) → dependência parcial → fere a 2FN. ✓

- Não é transitiva (3FN) nem atomicidade (1FN). **Correção:** Turma(Disciplina, Professor) + Matricula(Aluno, Disciplina).

Q4 — Mapeamento de generalização

`Pessoa` especializa-se em `PessoaFisica` (CPF) e `PessoaJuridica` (CNPJ), de forma **total e disjunta**. Deseja-se **evitar colunas nulas** e impedir que um cadastro seja PF e PJ.

Qual estratégia é a mais adequada?

- A) Uma tabela `Pessoa` com CPF e CNPJ (a maioria nula).
- B) Uma **tabela por subtipo**, cada uma com os atributos comuns.
- C) `Pessoa` + subtabelas, permitindo ser PF **e** PJ.
- D) `Pessoa` com listas de CPF/CNPJ por vírgula.

Q4 — Gabarito: B

Relembrando: especialização **total + disjunta** → **uma tabela por subtipo** elimina nulos e garante a exclusividade. 

- **A** gera muitos nulos; **C** permitiria sobreposição; **D** fere a 1FN.

Q5 — Isolamento

No sistema bancário, T1 lê o saldo; antes de T1 terminar, T2 **credita e confirma**; ao **reler**, T1 obtém **valor diferente**.

A anomalia é:

- A) Leitura suja (*dirty read*).
- B) Perda de atualização (*lost update*).
- C) Leitura fantasma (*phantom*).
- D) Leitura não repetível (*non-repeatable read*).

Q5 — Gabarito: D

Relembrando: reler a **mesma linha** e achar valor diferente (por *update confirmado* no meio) = **leitura não repetível**. ✓

- *Dirty* = ler não-confirmado; *phantom* = novas linhas; *lost update* = escritas se sobrepõem. Evita-se com `REPEATABLE READ`.

Q6 — Concorrência

Com 2PL, T1 trava A e T2 trava B; então T1 pede B e T2 pede A, ambas travadas.

Assinale a correta.

- A) O 2PL garante que deadlock nunca ocorra.
- B) Ocorre **deadlock**; o SGBD detecta, escolhe vítima e faz rollback.
- C) As duas concluem sem espera.
- D) É uma leitura fantasma.

Q6 — Gabarito: B

Relembrando: 2PL garante **serialização**, mas **não evita deadlock**. Espera circular → o SGBD **aborta uma vítima** (rollback) e a outra segue. 

 Prevenção por timestamp: *wait-die* (esperar-morrer) / *wound-wait* (ferir-esperar).

Q7 — Recuperação (WAL)

Após falha de energia: T1 deu commit **antes** da falha (páginas talvez não gravadas); T2 estava **ativa** (sem commit).

A recuperação aplica:

- A) UNDO em T1, REDO em T2.
- B) REDO em ambas.
- C) REDO em T1, UNDO em T2.
- D) Nada; o commit de T1 dispensa recuperação.

Q7 — Gabarito: C

Relembrando (WAL): confirmadas → **REDO** (durabilidade, caso de T1); não-confirmadas → **UNDO** (atomicidade, caso de T2). 

- O **checkpoint** limita o quanto o REDO precisa retroceder.

Q8 — Índices

Sobre uma tabela de **milhões de linhas**, a equipe decide onde criar índices. Em qual caso o índice **tende a NÃO ajudar**?

- A) Coluna `cpf` única em `WHERE cpf = ...`.
- B) FK usada em `JOIN`.
- C) Coluna **booleana** cujo filtro retorna **metade** das linhas.
- D) Coluna de data em faixas (`BETWEEN`).

Q8 — Gabarito: C

Relembrando: índice compensa quando a busca é **seletiva**. Coluna de **baixa seletividade** que retorna metade da tabela → o otimizador prefere **varredura completa**; o índice só onera escrita/espço. 

Q9 — NoSQL e CAP

Um **carrinho de compras global** precisa ficar **disponível e aceitando escritas** mesmo com **partições de rede**, tolerando inconsistência **temporária**.

Segundo o CAP, a escolha adequada é:

- A) **AP** com consistência eventual (ex.: banco de documentos distribuído).
- B) **CA** garantindo as três propriedades.
- C) Relacional com bloqueio global síncrono.
- D) **CP**, recusando escritas durante a partição.

Q9 — Gabarito: A

Relembrando (CAP): sob **partição**, escolhe-se **C ou A**. O carrinho quer **disponibilidade + escrita** → **AP** (modelo **BASE**, consistência eventual). 

- **B** é impossível sob partição; **D** recusaria escritas; **C** não escala.

Q10 — Arquitetura

A equipe **reorganiza o armazenamento físico** e **cria índices** de uma tabela, **sem alterar** aplicações nem o esquema conceitual.

Na ANSI-SPARC, isso é:

- A) Independência lógica.
- B) Independência física.
- C) Ausência de independência.
- D) Independência referencial.

Q10 — Gabarito: B

Relembrando: mudar o **nível interno** (físico/índices) sem afetar o conceitual/aplicações = **independência física**. 

- A **lógica** seria mudar o **esquema conceitual** sem afetar as **views** — não é o caso.

Fim da revisão de Banco de Dados

Próximo: mais Esquentas e o Aulão de BD (23/09).

Gabarito: 1A · 2B · 3C · 4B · 5D · 6B · 7C · 8C · 9A · 10B

 **Aulão (23/09):** resolveremos questões em **grupos** — traga suas dúvidas das unidades acima.